

```
class Banner
```

```
attr_accessible :horiz, :link, :visible, :image, :position  
has_attached_file :image, styles: { vert: '220'
```

```
before_create :assign_position
```

```
class Banner < ActiveRecord::Base
```

```
attr_accessible :horiz, :link, :visible, :image  
has_attached_file :image, styles: { vert: '220'
```

```
before_create :assign_position
```

```
protected
```

```
def assign_position
```

```
max = Banner.maximum(:position)
```

```
self.position = max ? max + 1 : 0
```

FULL STACK DEVELOPMENT

REACT - ADVANCED

AULA 05

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON.....	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	14
REFERÊNCIAS.....	15

O QUE VEM POR AÍ?

Prepare-se para elevar suas habilidades em React a um novo patamar com a nossa aula sobre testes de unidade! Imagine ter a confiança de que cada componente do seu projeto funciona perfeitamente, que cada interação do usuário é tratada corretamente, e que qualquer mudança no código não introduzirá novos erros. É exatamente isso que você vai conquistar ao aprender a configurar e utilizar Vitest e React Testing Library.

Nesta aula, mergulharemos de cabeça no mundo dos testes de unidade, começando pela configuração do ambiente de testes com Vitest e jsdom. Em seguida, você aprenderá a escrever testes que garantem que seus componentes React estão sendo renderizados corretamente e reagindo adequadamente às interações do usuário. Exploraremos a simulação de entrada do usuário, validação de formulários e verificação de mensagens de erro, assegurando que a experiência do usuário final seja sempre a melhor possível.

Mas não para por aí! Você também descobrirá como simular requisições HTTP e validar as respostas do servidor usando mocks com axios, garantindo que suas integrações de backend funcionem de maneira suave e sem falhas. Com esses conhecimentos, você será capaz de construir aplicações mais robustas, testáveis e confiáveis.

HANDS ON

Prepare-se para uma nova e empolgante etapa no desenvolvimento de suas habilidades com React! Nesta aula, mergulharemos no mundo dos testes de unidade, usando ferramentas poderosas como Vitest e React Testing Library. Se você já enfrentou desafios ao garantir que seu código funciona como esperado em diferentes cenários, esta aula será um divisor de águas.

Vamos começar configurando o Vitest e o jsdom, que nos permitirão simular o ambiente de um navegador e executar testes de forma eficiente. Você aprenderá como configurar o ambiente de testes e escrever testes básicos para garantir que seus componentes React estão renderizando corretamente e respondendo às interações do usuário como esperado.

No segundo vídeo, avançaremos criando testes mais sofisticados para validar a funcionalidade dos campos de formulário. Exploraremos como simular a entrada do usuário e verificar a validação do formulário, assegurando que mensagens de erro apropriadas sejam exibidas quando dados inválidos são inseridos. Isso é crucial para garantir uma experiência de usuário robusta e amigável.

Finalmente, no terceiro vídeo, introduziremos a simulação de requisições HTTP usando mocks com axios. Você verá como simular chamadas de API e validar que seus componentes lidam corretamente com respostas de servidor, uma habilidade essencial para aplicações que dependem de comunicação com backend.

SAIBA MAIS

De olho no Vitest

Vitest é uma ferramenta de testes unitários projetada para funcionar perfeitamente com o Vite, um bundler moderno e de alto desempenho para projetos JavaScript, especialmente aqueles que utilizam React. Desenvolvido com a mesma filosofia de Vite, o Vitest visa oferecer uma experiência de desenvolvimento rápida e eficiente, aproveitando ao máximo as vantagens do Vite. Mas por que exatamente precisamos do Vitest em um projeto React que usa Vite como bundler? Vamos explorar isso a seguir.

- **Integração nativa com Vite:** o principal benefício do Vitest é sua integração nativa com o Vite. Ele foi projetado para funcionar harmoniosamente com a configuração de Vite, utilizando o mesmo sistema de build. Isso significa que você não precisa configurar uma ferramenta de testes separada ou ajustar seu bundler para rodar testes, simplificando significativamente o setup inicial.
- **Velocidade e eficiência:** assim como o Vite, o Vitest é incrivelmente rápido. Ele utiliza o esbuild, que é conhecido por sua velocidade de compilação, permitindo feedback quase instantâneo durante o desenvolvimento. Isso torna o ciclo de desenvolvimento e teste muito mais eficiente, permitindo que você encontre e corrija erros rapidamente.
- **Ambiente de teste completo:** o suporte ao jsdom no Vitest permite que você simule um ambiente de navegador real. Isso é crucial para testar componentes React que dependem do DOM, garantindo que seu código funcione como esperado quando executado em um navegador real.
- **API intuitiva e poderosa:** Vitest fornece uma API poderosa e fácil de usar que permite escrever testes unitários, testes de integração e mocks de maneira intuitiva. A API é semelhante a outras bibliotecas de testes populares, o que facilita a adoção por pessoas desenvolvedoras que já estão familiarizadas com ferramentas como Jest.
- **Integração com React Testing Library:** você pode facilmente integrar a React Testing Library para testar componentes React de maneira declarativa. Isso permite que você escreva testes que se concentrem no comportamento do usuário e nas interações, em vez de detalhes de implementação.

Vitest vs. Jest: qual ferramenta escolher para seus testes em projetos React?

Quando se trata de testar aplicações React, duas das ferramentas mais proeminentes que você pode considerar são Vitest e [Jest](#). Cada uma delas traz suas próprias vantagens e características únicas, adaptando-se a diferentes tipos de projetos e necessidades específicas. Vamos explorar porque essas ferramentas são importantes e como elas podem se encaixar no seu fluxo de desenvolvimento.

Vitest é uma ferramenta relativamente nova no cenário de testes, desenvolvida para ser utilizada em conjunto com o Vite, um bundler moderno e extremamente rápido. O grande diferencial do Vitest é sua integração nativa com o Vite. Se você já está usando o Vite em seu projeto React, o Vitest se encaixa de maneira natural, aproveitando o mesmo sistema de build rápido e eficiente. Isso significa que você pode configurar seus testes sem grandes ajustes, mantendo a velocidade e a fluidez do desenvolvimento.

Por outro lado, Jest é uma ferramenta de testes bem estabelecida e amplamente utilizada, desenvolvida pelo Facebook. O Jest é conhecido por sua facilidade de uso e configuração zero, o que permite que desenvolvedores(as) comecem a testar rapidamente, sem a necessidade de ajustes complexos. Uma das principais características do Jest é seu suporte nativo para mocking, o que permite simular funções, módulos e temporizadores com facilidade, essencial para testar componentes React em isolamento. Além disso, o Jest oferece suporte para snapshot testing, que captura o estado da interface em um determinado momento e facilita comparações automáticas em testes futuros, garantindo que mudanças não introduzam regressões inesperadas.

Agora, você deve estar se perguntando: qual dessas ferramentas é a melhor para o meu projeto?

A resposta depende das necessidades específicas e do contexto do seu projeto. O Vitest, por exemplo, oferece uma vantagem significativa em termos de desempenho. Utilizando o esbuild, ele garante tempos de execução rápidos, proporcionando feedback quase instantâneo durante o desenvolvimento. Isso é particularmente útil em projetos que já utilizam o Vite, onde a integração é praticamente sem fricções.

Por outro lado, se o seu projeto se beneficia de funcionalidades avançadas como snapshot testing e uma ampla gama de plugins e extensões, o Jest pode ser a escolha mais adequada. O Jest possui uma grande comunidade de profissionais de desenvolvimento, o que significa que há muitos recursos, tutoriais e suporte disponíveis. Além disso, ele oferece uma excelente integração com TypeScript, facilitando o teste de aplicações que utilizam essa linguagem.

A escolha entre Vitest e Jest deve ser guiada pelas necessidades específicas do seu projeto. Se você valoriza desempenho e integração fluida com o Vite, o Vitest é uma escolha excelente. Se você precisa de funcionalidades avançadas e uma configuração simples, o Jest pode ser mais apropriado. Ambos são ferramentas poderosas que podem elevar a qualidade e a confiabilidade do seu código, garantindo que sua aplicação React esteja sempre funcionando como deveria.

userEvent vs. fireEvent: escolhendo a ferramenta certa para simular eventos no React Testing Library

No ecossistema do React Testing Library (RTL), duas ferramentas principais são utilizadas para simular eventos de usuário: o `userEvent` e o `fireEvent`. Ambas desempenham papéis importantes no teste de interações do usuário com componentes React, mas há razões claras pelas quais o `userEvent` é frequentemente a preferência entre os(as) desenvolvedores(as). Exploraremos o que cada uma dessas ferramentas oferece e por que o `userEvent` é geralmente a escolha preferida.

O `fireEvent` é a ferramenta original do RTL para simulação de eventos. Ela permite disparar eventos diretamente nos elementos do DOM. Com `fireEvent`, você pode simular cliques, mudanças de valor, teclas pressionadas e outros eventos nativos do DOM. A sintaxe é direta e poderosa, permitindo uma grande flexibilidade nos testes:

```
import { render, screen, fireEvent } from '@testing-library/react';
import MyComponent from './MyComponent';

test('simula clique usando fireEvent', () => {
  render(<MyComponent />);
```

```
fireEvent.click(screen.getByText('Enviar'));  
  
expect(screen.getByText('Obrigado!')).toBeInTheDocument();  
});
```

Embora o `fireEvent` seja muito útil, ele trabalha em um nível mais baixo, simulando diretamente os eventos no DOM, o que nem sempre representa de maneira realista as interações do usuário. É aqui que o `userEvent` entra em cena.

O `userEvent` é uma API de alto nível fornecida pelo RTL, projetada para simular interações do usuário de maneira mais realista. Enquanto `fireEvent` dispara eventos imediatamente, o `userEvent` simula a sequência natural de ações que um usuário realizaria, como digitar texto em um campo de entrada ou clicar em um botão. Isso inclui tempos de espera entre digitações e eventos `blur/focus`, criando uma simulação mais fiel do comportamento do usuário.

```
import { render, screen } from '@testing-library/react';  
import userEvent from '@testing-library/user-event';  
import MyComponent from './MyComponent';  
  
test('simula clique usando userEvent', async () => {  
  render(<MyComponent />);  
  await userEvent.click(screen.getByText('Enviar'));  
  
  expect(screen.getByText('Obrigado!')).toBeInTheDocument();  
});
```

A principal vantagem do `userEvent` sobre o `fireEvent` é a capacidade de representar interações do usuário de maneira mais natural e precisa. Isso é especialmente importante em testes de integração e de ponta a ponta, onde a precisão

na simulação do comportamento do usuário pode revelar problemas que testes com fireEvent poderiam não detectar.

Além disso, o userEvent cuida de muitos detalhes automaticamente, como gerenciamento de foco e tempos de espera, reduzindo a necessidade de código adicional para simular essas condições manualmente. Isso não só torna os testes mais fáceis de escrever, mas também mais robustos e confiáveis.

Embora o fireEvent ainda seja útil para casos específicos onde você precisa de controle direto sobre eventos DOM, o userEvent é geralmente preferido para simular interações do usuário de forma mais realista e intuitiva. Isso resulta em testes que são mais representativos de como os usuários realmente interagem com a aplicação, aumentando a qualidade e a confiabilidade do seu código.

Roles - Porque acessibilidade conta!

Realizar as buscas pelas roles é uma boa prática porque, além de testar a sua aplicação, você garante a sua acessibilidade. As especificações relacionadas à acessibilidade estão definidas na W3C (World Wide Web Consortium) como WAI-ARIA.

WAI-ARIA quer dizer Accessible Rich Internet Applications (Aplicações Ricas para uma Internet Acessível). O conjunto ARIA oferece a maneira de tornar as aplicações mais acessíveis a uma maior diversidade de pessoas, incluindo quem utiliza tecnologias assistivas, como leitores de telas e lentes de aumento.

[Aqui](#) você confere uma lista de todas as possíveis roles.

RTL: outras formas de buscar elementos

No decorrer dessa aula nós utilizamos algumas formas que a React Testing Library nos permite buscar os elementos HTML. Porém, existem diversas formas de realizar buscas no DOM.

Você encontrará, nesta aula, algumas que pode precisar, mas não deixe de conferir a [documentação](#) se precisar de algo mais específico.

Esses são os tipos de consultas (em inglês, queries) disponíveis, de acordo com a quantidade de elementos que você espera:

Buscando elementos únicos

- getBy;
- queryBy;
- findBy.

Buscando múltiplos elementos

- getAllBy
- queryAllBy
- findAllBy

E, aqui, você consegue ver as consultas específicas:

ByRole

getByRole, queryByRole, getAllByRole, queryAllByRole, findByRole, findAllByRole

Quando conseguimos encontrar elementos por sua role, significa que leitores de tela também serão capazes, e a aplicação estará acessível.

ByLabelText

getByLabelText, queryByLabelText, getAllByLabelText, queryAllByLabelText, findByLabelText, findAllByLabelText

É uma alternativa quando, por exemplo, temos múltiplos inputs no mesmo componente. Nesse caso, podemos buscar o input pelo label associado a ela (como fizemos em aula). Confira a seguir boas práticas para esse cenário:

```
<label for="email-input">E-mail</label>
<input id="email-input" />

<label id="email-label">E-mail</label>
```

```
<input aria-labelledby="email-label" />

<label>E-mail <input /></label>

<label>
  <span>E-mail</span>
  <input />
</label>

<input aria-label="E-mail" />
```

Em qualquer um dos exemplos acima, podemos obter o input da seguinte forma:

```
const inputEmail = screen.getByLabelText('E-mail')
```

ByPlaceholderText

getByPlaceholderText, queryByPlaceholderText, getAllByPlaceholderText, queryAllByPlaceholderText, findByPlaceholderText, findAllByPlaceholderText

Aqui, estamos falando do atributo do html, que também podemos utilizar para buscar os elementos:

```
<input placeholder="E-mail" />
```

E para buscar:

```
const inputEmail = screen.getByPlaceholderText('E-mail')
```

ByText

getText, queryByText, getAllByText, queryAllByText, findByText, findAllByText

Agora, estamos falando do `textContent` dos elementos HTML. Então, para buscar uma âncora com essa marcação:

```
<a href="/sobre">Sobre a Alura</a>
```

Escrevemos:

```
const ancoraSobre = screen.getByText('Sobre a Alura')
```

É comum também o uso de Expressões Regulares para fazer buscas por texto. Essas expressões são tão poderosas e úteis que existe até um [artigo na Alura](#) sobre elas.

ByDisplayValue

getByDisplayValue, queryByDisplayValue, getAllByDisplayValue, queryAllByDisplayValue, findByDisplayValue, findAllByDisplayValue

Podemos utilizar essa query para encontrar elementos do tipo `input`, `textarea` ou `select`.

O “display value” é o valor em si do elemento.

Vejamos um exemplo:

```
<input type="text" id="nome" value=' 'Agarikov' ' />
```

Agora com o valor preenchido, podemos realizar a query:

```
const nomeInput = screen.getByDisplayValue('Agarikov')
```

ByTestId

getByTestId, queryByTestId, getAllByTestId, queryAllByTestId, findByTestId, findAllByTestId

Quando tudo mais falhar, podemos definir um atributo data-testid no elemento e utilizar esse tipo de query para encontrá-lo:

```
<div data-testid="id-customizado" />
const elemento = screen.getByTestId('id-customizado')
```

Mas preste atenção nisto, pois nestes exemplos estamos sem valor semântico nenhum. Isso prejudica leitores de tela e demais softwares de acessibilidade.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, embarcamos em uma jornada para aprofundar nossas habilidades de teste de unidade em aplicações React, utilizando ferramentas poderosas como Vitest e React Testing Library (RTL). Esta experiência foi projetada para garantir que seu código funcione corretamente em diferentes cenários, oferecendo maior confiança na qualidade e robustez de suas aplicações.

Começamos com a configuração do Vitest e do jsdom, essenciais para simular o ambiente de um navegador e executar testes de forma eficiente. Este passo inicial foi crucial para estabelecer um ambiente de testes robusto, onde você aprendeu a configurar as ferramentas necessárias e a escrever testes básicos. Esses testes garantem que seus componentes React estão renderizando corretamente e respondendo às interações do usuário como esperado.

Avançando, nos aprofundamos na criação de testes mais sofisticados para validar a funcionalidade dos campos de formulário. Exploramos como simular a entrada do usuário e verificar a validação do formulário, assegurando que mensagens de erro apropriadas sejam exibidas quando dados inválidos são inseridos. Essa parte da aula foi fundamental para garantir uma experiência de usuário robusta e amigável, validando que os componentes lidam corretamente com diferentes tipos de entrada.

Por fim, introduzimos a simulação de requisições HTTP usando mocks com axios. Esta técnica permite que você simule chamadas de API e valide como seus componentes lidam com as respostas do servidor. Esse conhecimento é essencial para aplicações que dependem de comunicação com backend, garantindo que seu código funcione bem não apenas em um ambiente isolado, mas também quando interage com serviços externos.

Ao final desta aula, você adquiriu uma compreensão sólida de como configurar e utilizar Vitest e React Testing Library para testar suas aplicações React. Aprendeu a criar um ambiente de testes eficiente, a validar a funcionalidade dos componentes e a simular interações complexas com o backend. Com essas habilidades, você está melhor preparado(a) para garantir a qualidade e a confiabilidade de suas aplicações, elevando suas práticas de desenvolvimento a um novo patamar.

REFERÊNCIAS

JEST. **Getting Started**. [s.d]. Disponível em: <https://jestjs.io/docs/getting-started>. Acesso em: 26 jun. 2024.

TESTING LIBRARY. **Introduction**. [s.d]. Disponível em: <https://testing-library.com/docs/ecosystem-user-event>. Acesso em: 26 jun. 2024.

TESTING LIBRARY. **React Testing Library**. [s.d]. Disponível em: <https://testing-library.com/docs/react-testing-library/intro/>. Acesso em: 26 jun. 2024.

VITEST. [s.d]. Disponível em: <https://vitest.dev/>. Acesso em: 26 jun. 2024.

PALAVRAS-CHAVE

Vitest, Jest, userEvent, fireEvent, React Testing Library, testes unitários, simulação de eventos, desenvolvimento React.

EXEMPLO

POSTECH